

Министерство науки и высшего образования РФ  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

ОТЧЕТ  
по лабораторной работе №3  
по курсу «Логика и основы алгоритмизации  
в инженерных задачах»  
на тему «Динамические списки»

Выполнили ст. гр. 25BBB2:

Карпов А.А.

Долгов Л.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

## **Тема: Динамические списки**

**Цель работы:** изучить организацию динамических структур данных на основе однонаправленного списка и получить практические навыки реализации на его основе абстрактных структур данных — очереди с приоритетом, очереди и стека.

**Лабораторные задания:** 1) реализовать очередь с приоритетом, вставляя объекты в список в соответствии с их приоритетом (обязательное задание); 2\*) реализовать на основе предложенного кода структуру «очередь» (Queue); 3\*) реализовать на основе предложенного кода структуру «стек» (Stack).

**Средства реализации и проверки:** программы реализованы на языке Си с использованием динамического выделения памяти (malloc/free); корректность работы каждой программы проверялась многократным запуском с разными входными данными, а отсутствие утечек памяти — анализатором valgrind.

## **Теоретические сведения**

Однонаправленный (односвязный) список — это динамическая структура данных, состоящая из узлов, каждый из которых хранит информационное поле и указатель на следующий узел списка. В отличие от массива, память под узлы списка выделяется по одному в процессе работы программы (функцией malloc), а не заранее единым блоком, поэтому размер списка ограничен только объёмом доступной памяти и заранее не фиксируется. Доступ к списку осуществляется через указатель на его первый узел — «голову»; последний узел списка содержит в поле указателя значение NULL, обозначающее конец списка.

Базовый набор операций над односвязным списком включает создание нового узла с вводом данных пользователем, добавление узла в список, последовательный просмотр (обход) всех узлов списка от головы до конца, поиск узла по заданному критерию и удаление узла с освобождением занятой им памяти. Именно на основе этого набора операций и строятся более специализированные абстрактные структуры данных, рассматриваемые в настоящей работе: приоритетная очередь, очередь и стек — каждая отличается

только правилом, по которому новый элемент добавляется в список и по которому выбирается элемент для извлечения.

## **Задание 1. Приоритетная очередь**

### **Организация структуры**

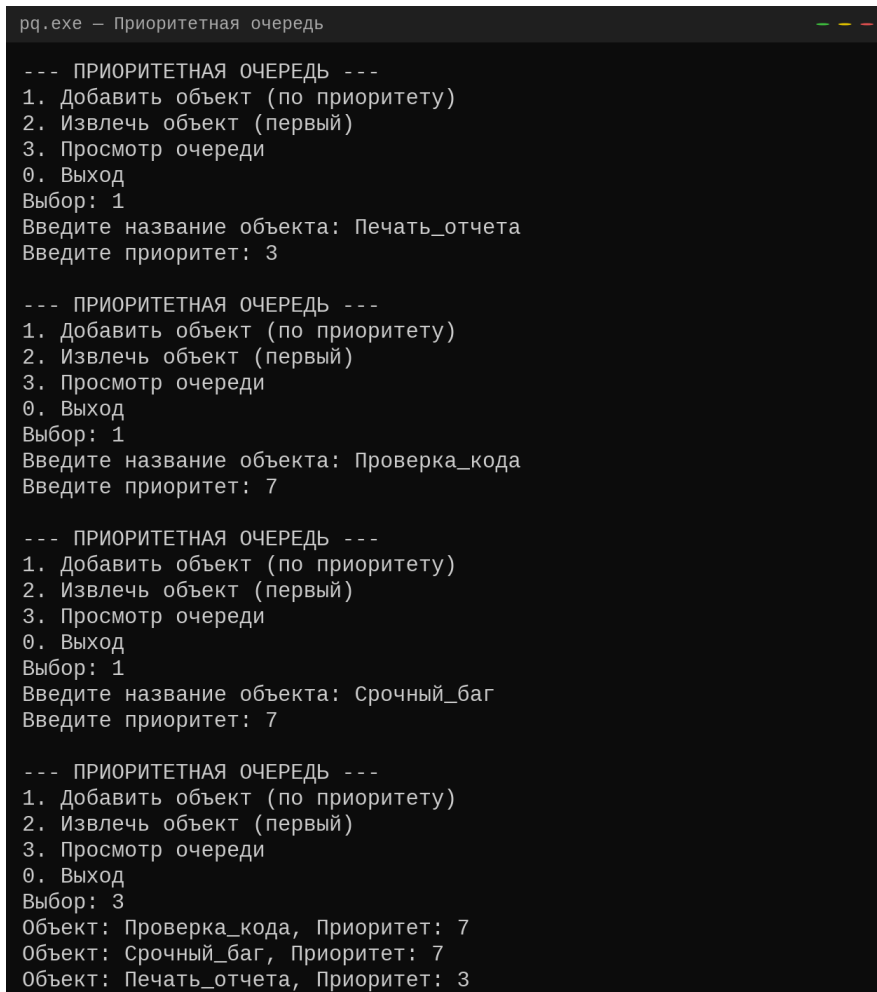
Приоритетная очередь строится на основе того же односвязного списка, но в информационную часть узла дополнительно введено целочисленное поле «приоритет». Список поддерживается в отсортированном по убыванию приоритета виде: чем важнее объект, тем ближе к голове списка он расположен. Функция добавления нового элемента сначала сравнивает его приоритет с приоритетом головного узла — если новый элемент важнее всех уже находящихся в очереди, он становится новой головой; в противном случае список просматривается от начала до тех пор, пока не будет найдена позиция, следующий узел за которой имеет приоритет строго ниже приоритета вставляемого элемента, после чего новый узел подключается в найденное место. При равенстве приоритетов новый элемент вставляется после всех уже имеющихся элементов с тем же приоритетом, то есть при равных приоритетах очередь ведёт себя как обычная FIFO-очередь — это обеспечивает предсказуемое и справедливое поведение при совпадении приоритетов. Извлечение всегда происходит с головы списка — так из очереди забирается объект с наивысшим приоритетом, а если таких несколько — тот из них, что был добавлен раньше.

Вставка одного элемента в худшем случае требует полного прохода по списку, то есть имеет сложность  $O(n)$ , где  $n$  — число элементов в очереди на момент вставки; извлечение элемента с головы списка выполняется за  $O(1)$ .

### **Проверка работы программы**

Программа предоставляет текстовое меню с операциями добавления объекта (с вводом его названия и приоритета), извлечения объекта с наивысшим приоритетом и просмотра текущего содержимого очереди. Для проверки в очередь было добавлено три объекта: «Печать\_отчета» с приоритетом 3, «Проверка\_кода» с приоритетом 7 и «Срочный\_баг» также с приоритетом 7 —

последние два специально выбраны равными, чтобы проверить сохранение порядка вставки при совпадении приоритетов. Результат добавления и последующего просмотра очереди показан на рисунке 1.



```
rq.exe — Приоритетная очередь

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Печать_отчета
Введите приоритет: 3

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Проверка_кода
Введите приоритет: 7

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Срочный_баг
Введите приоритет: 7

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: Проверка_кода, Приоритет: 7
Объект: Срочный_баг, Приоритет: 7
Объект: Печать_отчета, Приоритет: 3
```

Рисунок 1 — Формирование приоритетной очереди

Как видно из рисунка 1, оба объекта с приоритетом 7 расположились впереди объекта с приоритетом 3, при этом «Проверка\_кода», добавленный раньше «Срочного\_бага», занял более раннюю позицию — порядок вставки при равных приоритетах сохранён. Далее оба объекта с наивысшим приоритетом были последовательно извлечены, после чего очередь была снова просмотрена; результат показан на рисунке 2.

```
rq.exe — Приоритетная очередь

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 2
Извлечен объект: Проверка_кода (Приоритет: 7)

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 2
Извлечен объект: Срочный_баг (Приоритет: 7)

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: Печать_отчета, Приоритет: 3

--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 0
```

Рисунок 2 — Извлечение объектов из приоритетной очереди

Извлечение выполняется строго в порядке убывания приоритета: первым был извлечён «Проверка\_кода», вторым — «Срочный\_баг», и в очереди остался только объект «Печать\_отчета» с приоритетом 3. Программа была дополнительно проверена анализатором `valgrind` — утечек памяти и обращений к освобождённой памяти не обнаружено, вся выделенная память корректно освобождается как при извлечении элементов, так и при выходе из программы.

## Задание 2\*. Очередь (Queue)

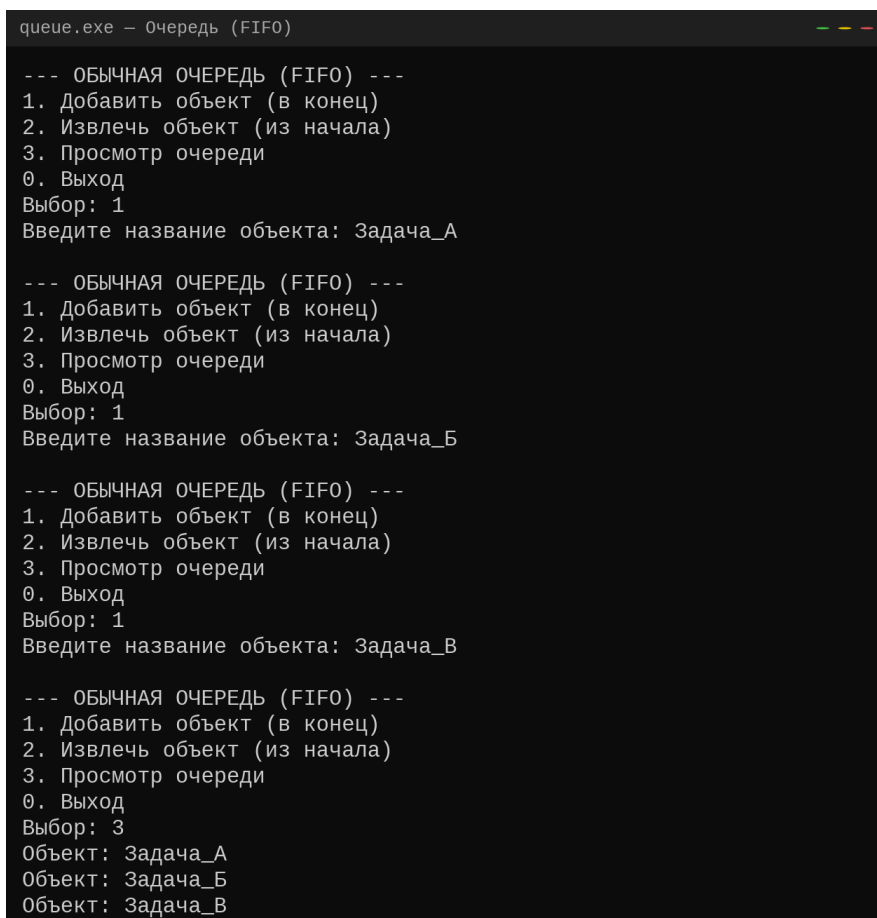
### Организация структуры

В отличие от приоритетной очереди, обычная очередь не учитывает никаких дополнительных признаков объектов и работает по правилу «первым пришёл — первым ушёл» (FIFO). Для того чтобы добавление нового элемента в конец списка выполнялось за  $O(1)$ , а не за  $O(n)$ , как потребовал бы поиск последнего узла при каждой вставке, в программе помимо указателя на голову списка введён и хранится отдельный указатель на последний узел. Новый элемент подключается к узлу, на который указывает этот указатель, после чего указатель переставляется на добавленный узел. Извлечение, как и в базовом

списке, происходит с головы: указатель на голову переставляется на следующий узел, а память удалённого узла освобождается; если после извлечения список опустел, указатель на последний узел также сбрасывается в NULL, чтобы не остаться висячим.

## Проверка работы программы

В очередь было последовательно добавлено три объекта: «Задача\_А», «Задача\_Б» и «Задача\_В». Результат добавления и просмотра очереди показан на рисунке 3.



```
queue.exe - Очередь (FIFO)

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Задача_А

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Задача_Б

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта: Задача_В

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: Задача_А
Объект: Задача_Б
Объект: Задача_В
```

Рисунок 3 — Формирование очереди

Объекты в очереди расположены в том же порядке, в котором добавлялись, — «Задача\_А» первой, «Задача\_В» последней. Далее из очереди был извлечён один объект и очередь снова просмотрена; результат показан на рисунке 4.

```
queue.exe — Очередь (FIFO)

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 2
Извлечен объект: Задача_A

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: Задача_Б
Объект: Задача_В

--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 0
```

Рисунок 4 — Извлечение объекта из очереди

Извлечённым оказался именно «Задача\_A» — первый добавленный объект, что подтверждает соблюдение дисциплины FIFO; в очереди остались «Задача\_Б» и «Задача\_В» в исходном порядке. Проверка valgrind утечек памяти не выявила.

### Задание 3\*. Стек (Stack)

#### Организация структуры

Стек, в отличие от очереди, работает по правилу «последним пришёл — первым ушёл» (LIFO), поэтому обе его операции — добавление и извлечение — выполняются с одного и того же конца списка, а именно с головы, и второй указатель (на последний узел) для его реализации не требуется. При добавлении новый узел подключается так, что его указатель next ссылается на прежнюю голову списка, после чего сам новый узел становится головой — он и есть «вершина» стека. При извлечении головной узел удаляется, а вершиной становится следующий за ним узел. Обе операции выполняются за  $O(1)$ .

#### Проверка работы программы

На стек было последовательно помещено три объекта: «Тарелка\_1», «Тарелка\_2» и «Тарелка\_3» — по аналогии со стопкой тарелок, для которой принцип LIFO нагляднее всего. Результат добавления и просмотра стека показан на рисунке 5.

```
stack.exe — Стек (LIFO)

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 1
Введите название объекта: Тарелка_1

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 1
Введите название объекта: Тарелка_2

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 1
Введите название объекта: Тарелка_3

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 3
Объект: Тарелка_3 <-- вершина
Объект: Тарелка_2
Объект: Тарелка_1
```

Рисунок 5 — Формирование стека

Вершиной стека, как и ожидалось, оказался последний добавленный объект — «Тарелка\_3». Далее один объект был снят со стека и стек снова просмотрен; результат показан на рисунке 6.

```
stack.exe — Стек (LIFO)

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 2
Извлечен объект: Тарелка_3

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 3
Объект: Тарелка_2 <-- вершина
Объект: Тарелка_1

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 0
```

Рисунок 6 — Извлечение объекта со стека

Снятым оказался именно «Тарелка\_3» — последний помещённый на стек объект, что подтверждает соблюдение дисциплины LIFO; новой вершиной стала «Тарелка\_2». Проверка valgrind утечек памяти также не выявила.

## **Вывод**

В ходе выполнения работы изучена организация динамических структур данных на основе однонаправленного списка и получены практические навыки реализации на его основе трёх абстрактных структур: приоритетной очереди, очереди и стека. Экспериментальная проверка всех трёх программ подтвердила корректность реализованной логики: приоритетная очередь правильно упорядочивает объекты по убыванию приоритета и сохраняет порядок вставки при равных приоритетах, обычная очередь соблюдает дисциплину FIFO, а стек — дисциплину LIFO. Проверка анализатором valgrind во всех трёх случаях не выявила утечек памяти, что говорит о корректном освобождении памяти узлов при извлечении элементов и при завершении работы программ.